HRMS

12 通知服務 — 需求分析書

HR12 NOTIFICATION Service

系統名稱	版本
人力資源暨專案管理系統	**1.0**
日期	作者
2026-03-18	**HRMS 開發團隊**

目錄

PM 審查補充

通知服務(Notification Service) 需求分析書

版本: 1.0 **所屬領域:** 通用領域 (Generic Domain) **導入階段:** 第一階段（核心基礎服務）

---

1. 服務概述

1.1 核心職責

- 多渠道通知（Email、系統訊息、推播、Teams/Slack/LINE）
- 通知範本管理
- 自動提醒事件（生日、合約到期、證照到期等）
- 通知偏好設定
- 訊息佇列處理（確保送達）

---

2. 領域模型

2.1 聚合根

Notification (通知)

```
Notification {
  notificationId: UUID
  recipientId: UUID (FK to Employee)

  title: String
  content: Text
  notificationType: NotificationType

  channels: List<NotificationChannel>

  priority: Priority (LOW, NORMAL, HIGH, URGENT)

  status: NotificationStatus (PENDING, SENT, FAILED, READ)

  sentAt: DateTime (nullable)
  readAt: DateTime (nullable)

  relatedBusinessType: String (nullable, leave, overtime...)
  relatedBusinessId: UUID (nullable)

  createdAt: DateTime
}

enum NotificationType {
  APPROVAL_REQUEST     // 審核請求
  APPROVAL_RESULT      // 審核結果
  REMINDER            // 提醒
  ANNOUNCEMENT        // 公告
  ALERT               // 警示
}

enum NotificationChannel {
  IN_APP     // 系統內訊息
  EMAIL      // Email
  PUSH       // 推播
  TEAMS      // Microsoft Teams
  SLACK      // Slack
  LINE       // LINE
}

enum Priority {
  LOW
  NORMAL
  HIGH
```

```
  URGENT
}

enum NotificationStatus {
  PENDING
  SENT
  FAILED
  READ
}
```

NotificationTemplate (通知範本)

```
NotificationTemplate {
  templateId: UUID
  templateCode: String (unique)
  templateName: String

  subject: String (支援變數 {{name}})
  body: Text (支援變數)

  defaultChannels: List<NotificationChannel>

  isActive: Boolean
  createdAt: DateTime
}
```

---

3. Event-Driven架構

3.1 訂閱業務事件（關鍵！）

Notification Service訂閱**所有**業務事件，根據事件類型發送通知：

業務事件	通知對象	通知內容
`LeaveApplied`	直屬主管	“員工{{name}}申請{{days}}天請假，請審核”
`LeaveApproved`	申請人	“您的請假申請已核准”
`LeaveRejected`	申請人	“您的請假申請被駁回，原因:{{reason}}”
`OvertimeApplied`	直屬主管	“員工{{name}}申請{{hours}}小時加班”
`UserLocked`	被鎖定使用者	“您的帳號已被鎖定，請聯絡系統管理員”
`PayslipGenerated`	員工	“您的{{month}}月薪資單已產生”
`AnnualLeaveExpiring`	員工+主管	“員工{{name}}有{{days}}天特休將於{{date}}到期”
`ContractExpiring`	HR	“員工{{name}}的合約將於{{date}}到期”
`CertificateExpiring`	員工	“您的{{cert}}證照將於{{date}}到期”

4. 核心API

4.1 發送通知API（內部服務調用）

```
POST /api/v1/notifications/send
Request:
{
  "recipientIds": ["uuid1", "uuid2"],
  "templateCode": "LEAVE_APPROVED",
  "variables": {
    "employeeName": "張三",
    "leaveDays": 2,
    "leaveType": "特休"
  },
  "channels": ["IN_APP", "EMAIL"],
  "priority": "NORMAL"
}

Response 202 Accepted:
{
  "notificationIds": ["uuid1", "uuid2"],
  "message": "通知已加入傳送佇列"
}
```

業務邏輯（異步）：

1. 根據templateCode載入範本
2. 替換變數生成最終內容
3. 加入訊息佇列
4. 依序發送各渠道
5. 更新發送狀態

4.2 查詢個人通知

```
GET /api/v1/notifications/me?status=UNREAD
Response 200:
[
  {
    "notificationId": "uuid",
    "title": "請假審核",
    "content": "員工張三申請2天請假，請審核",
    "type": "APPROVAL_REQUEST",
    "priority": "NORMAL",
    "status": "SENT",
    "sentAt": "2025-11-24T10:00:00Z",
    "relatedBusinessType": "leave",
    "relatedBusinessId": "uuid"
  }
]
```

4.3 標記為已讀

```
PUT /api/v1/notifications/{id}/read
Response 200:
{
  "notificationId": "uuid",
  "status": "READ",
  "readAt": "2025-11-24T11:00:00Z"
}
```

---

5. 多渠道實作

5.1 Email通知

- 技術: Spring Mail + SMTP
- 支援HTML格式
- 附件支援

5.2 系統內訊息

- 儲存於資料庫
- WebSocket推送即時通知
- 前端顯示小鈴鐺提示

5.3 推播通知

- 技術: Firebase Cloud Messaging (FCM)
- iOS/Android App支援

5.4 企業通訊軟體（選配）

- Teams: Teams Webhook
- Slack: Slack Bot API
- LINE: LINE Notify API

---

6. 自動提醒Job

6.1 定期排程任務

Job名稱	執行頻率	功能
BirthdayReminderJob	每日08:00	當日生日員工提醒
ContractExpiryJob	每日09:00	合約30天內到期提醒
AnnualLeaveExpiryJob	每週一	特休30天內到期提醒
CertificateExpiryJob	每週一	證照30天內到期提醒
TimesheetReminderJob	每日18:00	未回報工時提醒

---

7. 非功能需求

需求	目標
Email發送成功率	>98%
系統內訊息即時性	<3秒
推播送達率	>95%
訊息佇列處理能力	1000 msg/min

文件結束

PM審查補充

通知服務 - PM審查補充文件

版本: 1.1
日期: 2025-12-03
補充說明: 根據PM審查報告補充CROSS-001完整設計

📋 補充的PM審查項目

CROSS-001: 通知服務完整設計

- 通知偏好設定API
- 企業通訊軟體整合(Teams/Slack/LINE)
- 事件訂閱清單完整說明
- 通知範本管理

---

1. 通知偏好設定

1.1 UserNotificationPreference聚合根

```
UserNotificationPreference {
  userId: UUID

  // 通知渠道偏好（按事件類型）
  channelPreferences: Map<NotificationType, List<Channel>>

  // 勿擾時段
  quietHours: QuietHours

  // 語言偏好
  language: String (zh-TW, en-US)

  // 全域開關
  isEnabled: Boolean

  updatedAt: DateTime
}

QuietHours {
  enabled: Boolean
  startTime: Time (預設: 22:00)
  endTime: Time (預設: 08:00)
  daysOfWeek: List<DayOfWeek> (可指定特定日期)
}

enum NotificationType {
  LEAVE_APPLIED
  LEAVE_APPROVED
  OVERTIME_ALERT
  PAYSLIP_GENERATED
  PASSWORD_EXPIRING_SOON
  PROJECT_BUDGET_ALERT
  // ... 更多通知類型
}

enum Channel {
  EMAIL
  IN_APP      // 系統內通知
  PUSH        // 手機推播
  TEAMS       // Microsoft Teams
  SLACK
  LINE
}
```

1.2 預設偏好範例

```
{
  "userId": "uuid",
  "channelPreferences": {
    "LEAVE_APPROVED": ["EMAIL", "IN_APP", "PUSH"],
    "PAYSLIP_GENERATED": ["EMAIL"],
    "OVERTIME_ALERT": ["EMAIL", "IN_APP"],
    "PROJECT_BUDGET_ALERT": ["EMAIL", "TEAMS"]
  },
  "quietHours": {
    "enabled": true,
    "startTime": "22:00",
    "endTime": "08:00",
    "daysOfWeek": ["MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY"]
  },
  "language": "zh-TW",
  "isEnabled": true
}
```

1.3 API設計

```
PUT /api/v1/notifications/preferences
更新通知偏好

GET /api/v1/notifications/preferences
查詢個人通知偏好

POST /api/v1/notifications/test
測試通知（發送測試訊息驗證設定）
```

---

2. 企業通訊軟體整合

2.1 Microsoft Teams整合

Webhook設定

```
管理員配置：
1. 在Teams頻道中新增"傳入Webhook"連接器
2. 取得Webhook URL
3. 在系統中配置：
   - 組織層級：預設HR通知頻道
   - 部門層級：部門專屬頻道
   - 專案層級：專案工作頻道
```

發送實作

```java
@Service
public class TeamsNotificationService implements NotificationChannel {

    public void send(NotificationMessage message, String webhookUrl) {
        TeamsMessageCard card = TeamsMessageCard.builder()
            .title(message.getTitle())
            .summary(message.getSummary())
            .themeColor(getThemeColor(message.getPriority()))
            .sections(buildSections(message))
            .potentialAction(buildActions(message))
            .build();

        HttpHeaders header = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);

        HttpEntity<TeamsMessageCard> request = new HttpEntity<>(card, headers);

        restTemplate.postForEntity(webhookUrl, request, Void.class);
    }

    private String getThemeColor(Priority priority) {
        return switch (priority) {
            case URGENT -> "FF0000"; // 紅色
            case HIGH -> "FFA500";   // 橘色
            case NORMAL -> "0078D7";  // 藍色
            case LOW -> "808080";    // 灰色
        };
    }
}
```

訊息範例

```
{
  "@type": "MessageCard",
  "@context": "https://schema.org/extensions",
  "summary": "專案預算警示",
  "themeColor": "FFA500",
  "title": "⚠️ 專案預算警示",
  "sections": [
    {
      "activityTitle": "XX銀行專案",
      "facts": [
        {"name": "預算使用率", "value": "85%"},
        {"name": "累計成本", "value": "8,500,000元"},
        {"name": "預算總額", "value": "10,000,000元"}
      ],
      "text": "專案成本已達預算85%，請注意控管。"
    }
  ],
  "potentialAction": [
    {
      "@type": "OpenUri",
      "name": "查看專案詳情",
      "targets": [
        {
          "os": "default",
          "uri": "https://hr.company.com/projects/uuid"
        }
      ]
    }
  ]
}
```

2.2 LINE Notify整合

Access Token取得

```
1. 訪問 https://notify-bot.line.me/
2. 登入LINE帳號
3. 發行存取權杖(Access Token)
4. 選擇通知的聊天室/群組
5. 將Access Token儲存至系統配置
```

發送實作

```
public void sendLineNotification(String message, String accessToken) {
    String url = "https://notify-api.line.me/api/notify";

    HttpHeaders headers = new HttpHeaders();
    headers.setBearerAuth(accessToken);
    headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);

    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("message", message);

    HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<>(body, headers);

    restTemplate.postForEntity(url, request, Void.class);
}
```

訊息格式

```
\n您的請假申請已核准
假別：特休假
期間：2025-12-10 ~ 2025-12-12
天數：3天
核准人：李經理
剩餘假期：5天
```

2.3 Slack整合

Webhook配置

```
1. 在Slack App Directory中新增"Incoming Webhooks"
2. 選擇通知頻道
3. 取得Webhook URL
4. 配置至系統
```

發送實作

```
public void sendSlackNotification(NotificationMessage message, String webhookUrl) {
    SlackMessage slackMsg = SlackMessage.builder()
        .text(message.getTitle())
        .blocks(buildBlocks(message))
        .build();

    restTemplate.postForEntity(webhookUrl, slackMsg, Void.class);
}
```

---

3. 事件訂閱清單

3.1 完整事件訂閱表

業務事件	來源服務	通知對象	預設渠道	優先級	範本ID
`EmployeeCreated`	Organization	新員工	Email	NORMAL	TMPL_WELCOME
`LeaveApplied`	Attendance	主管	Email+系統	NORMAL	TMPL_LEAVE_APPLY
`LeaveApproved`	Attendance	申請人	Email+推播	NORMAL	TMPL_LEAVE_APPROVED
`OvertimeApproved`	Attendance	申請人	Email	NORMAL	TMPL_OT_APPROVED
`OvertimeLimitExceeded`	Attendance	HR+主管	Email+系統	HIGH	TMPL_OT_ALERT
`AnnualLeaveExpiring`	Attendance	員工	Email+系統	NORMAL	TMPL_LEAVE_EXPIRY
`PayslipGenerated`	Payroll	員工	Email	NORMAL	TMPL_PAYSLIP
`PayrollRunCompleted`	Payroll	HR	Email	NORMAL	TMPL_PAYROLL_DONE
`InsuranceEnrollmentCompleted`	Insurance	員工	Email	LOW	TMPL_INS_ENROLL
`PasswordExpiringSoon`	IAM	使用者	Email+系統	NORMAL	TMPL_PWD_EXPIRY
`PasswordExpired`	IAM	使用者	Email	HIGH	TMPL_PWD_EXPIRED
`ProjectBudgetAlert`	Project	PM+總監	Email+Teams	URGENT	TMPL_BUDGET_ALERT
`TimesheetApproved`	Timesheet	員工	系統	LOW	TMPL_TIME_APPROVED
`BenefitApplicationApproved`	Organization	員工	Email+推播	NORMAL	TMPL_BENEFIT_APPROVED
`ContractExpiring` 合約到期	Organization	HR	Email	NORMAL	TMPL_CONTRACT_EXPIRY

3.2 事件處理邏輯

```
@Service
@Slf4j
public class NotificationEventHandler {

    @KafkaListener(topics = "${kafka.topics.domain-events}")
    public void handleDomainEvent(DomainEvent event) {
        try {
            // 1. 識別事件類型
            String eventType = event.getEventType();

            // 2. 查找通知配置
            NotificationConfig config = configRepo.findByEventType(eventType);
            if (config == null) {
                return; // 無需通知的事件
            }

            // 3. 確定通知對象
            List<UUID> recipients = determineRecipients(event, config);

            // 4. 為每個接收者發送通知
            for (UUID recipientId : recipients) {
                // 查詢使用者偏好
                UserNotificationPreference pref = prefRepo.findByUserId(recipientId);

                // 檢查勿擾時段
                if (isQuietHours(pref)) {
                    scheduleForLater(event, recipientId); // 稍後發送
                    continue;
                }

                // 取得通知範本
                NotificationTemplate template = templateRepo.findById(config.getTemplateId(

                // 渲染訊息內容
                String content = templateEngine.render(template, event.getPayload());

                // 根據偏好選擇渠道發送
                List<Channel> channels = pref.getChannelPreferences().get(eventType);
                for (Channel channel : channels) {
                    sendViaChannel(channel, recipientId, content);
                }
            }

        } catch (Exception e) {
            log.error("處理通知事件失敗: {}", event, e);
            // 重試機制...
        }
```

```
    }
}
```

---

4. 通知範本管理

4.1 範本結構

```
NotificationTemplate {
  templateId: String (TMPL_LEAVE_APPROVED)
  templateName: String
  category: TemplateCategory

  subject: String (可包含變數)
  body: Text (可包含變數)

  variables: List<String> (可用變數清單)

  supportedChannels: List<Channel>

  isActive: Boolean
}
```

4.2 範本範例：請假核准通知

```
templateId: TMPL_LEAVE_APPROVED
subject: 請假申請核准通知 - {{leaveTypeName}}
body: |
  親愛的 {{employeeName}}，

  您申請的{{leaveTypeName}}已獲核准：

  假別：{{leaveTypeName}}
  期間：{{startDate}} ~ {{endDate}}
  天數：{{totalDays}}天
  核准人：{{approverName}}
  核准時間：{{approvedAt}}

  剩餘假期餘額：{{remainingDays}}天

  ---
  此為系統自動通知，請勿直接回覆。
  如有疑問請聯繫您的主管或HR部門。

variables: [
  employeeName, leaveTypeName, startDate, endDate,
  totalDays, approverName, approvedAt, remainingDays
]

supportedChannels: [EMAIL, IN_APP, PUSH]
```

4.3 變數替換引擎

```java
public String renderTemplate(NotificationTemplate template, Map<String, Object> data) {
    String content = template.getBody();

    for (Map.Entry<String, Object> entry : data.entrySet()) {
        String placeholder = "{{" + entry.getKey() + "}}";
        String value = entry.getValue().toString();
        content = content.replace(placeholder, value);
    }

    return content;
}
```

5. 業務案例

業務案例 UC-NOTIF-001: 多渠道通知實例

情境: 張三申請2天特休，李經理核准

步驟1: 事件發布

```
Attendance Service:
- 李經理點擊「核准」
- 發布LeaveApproved事件至Kafka
```

步驟2: Notification Service接收

```
- 訂閱Kafka topic
- 接收LeaveApproved事件
- 解析event payload:
  {
    "employeeId": "uuid-zhangsan",
    "employeeName": "張三",
    "leaveTypeName": "特休假",
    "startDate": "2025-12-10",
    "endDate": "2025-12-11",
    "totalDays": 2,
    "approverName": "李經理",
    "remainingDays": 5
  }
```

步驟3: 查詢通知偏好

```
查詢張三的NotificationPreference:
{
  "LEAVE_APPROVED": ["EMAIL", "IN_APP", "PUSH"]
}

檢查勿擾時段:
- 當前時間: 15:30
- 勿擾時段: 22:00-08:00
- 結果: 非勿擾時段，可立即發送 ✅
```

步驟4: 載入範本並渲染

```
範本ID: TMPL_LEAVE_APPROVED

渲染後內容:
主旨: 請假申請核准通知 - 特休假
內容:
  親愛的 張三，

  您申請的特休假已獲核准：
  假別：特休假
  期間：2025-12-10 ~ 2025-12-11
  天數：2天
  核准人：李經理
  剩餘假期餘額：5天
  ...
```

步驟5: 多渠道發送

```
Email渠道:
- 收件人: zhang.san@company.com
- 主旨: 請假申請核准通知 - 特休假
- 內容: (渲染後HTML)
- 發送狀態: ✅ 成功

系統內通知:
- 建立InAppNotification記錄
- 使用者登入時顯示通知鈴鐺
- 發送狀態: ✅ 成功

手機推播:
- 使用FCM (Firebase Cloud Messaging)
- 推送至張三的手機
- 發送狀態: ✅ 成功
```

步驟6: 記錄發送歷史

```
NotificationLog:
- 事件: LeaveApproved
- 接收者: 張三
- 渠道: EMAIL (成功), IN_APP (成功), PUSH (成功)
- 發送時間: 2025-12-03 15:30:00
```

補充文件結束

主文件: 12_通知服務需求分析書.md
修訂日期: 2025-12-03
修訂人: SA

PM 審查補充

工時管理、跨服務整合、支援服務 - PM審查補充文件

版本: 1.1
日期: 2025-12-03
涵蓋服務: 工時管理(07)、績效/招募/訓練(08-10)、簽核流程(11)、通知(12)、文件(13)、報表分析(14)

📋 本文件補充的PM審查項目

CROSS-001: 通知服務完整設計

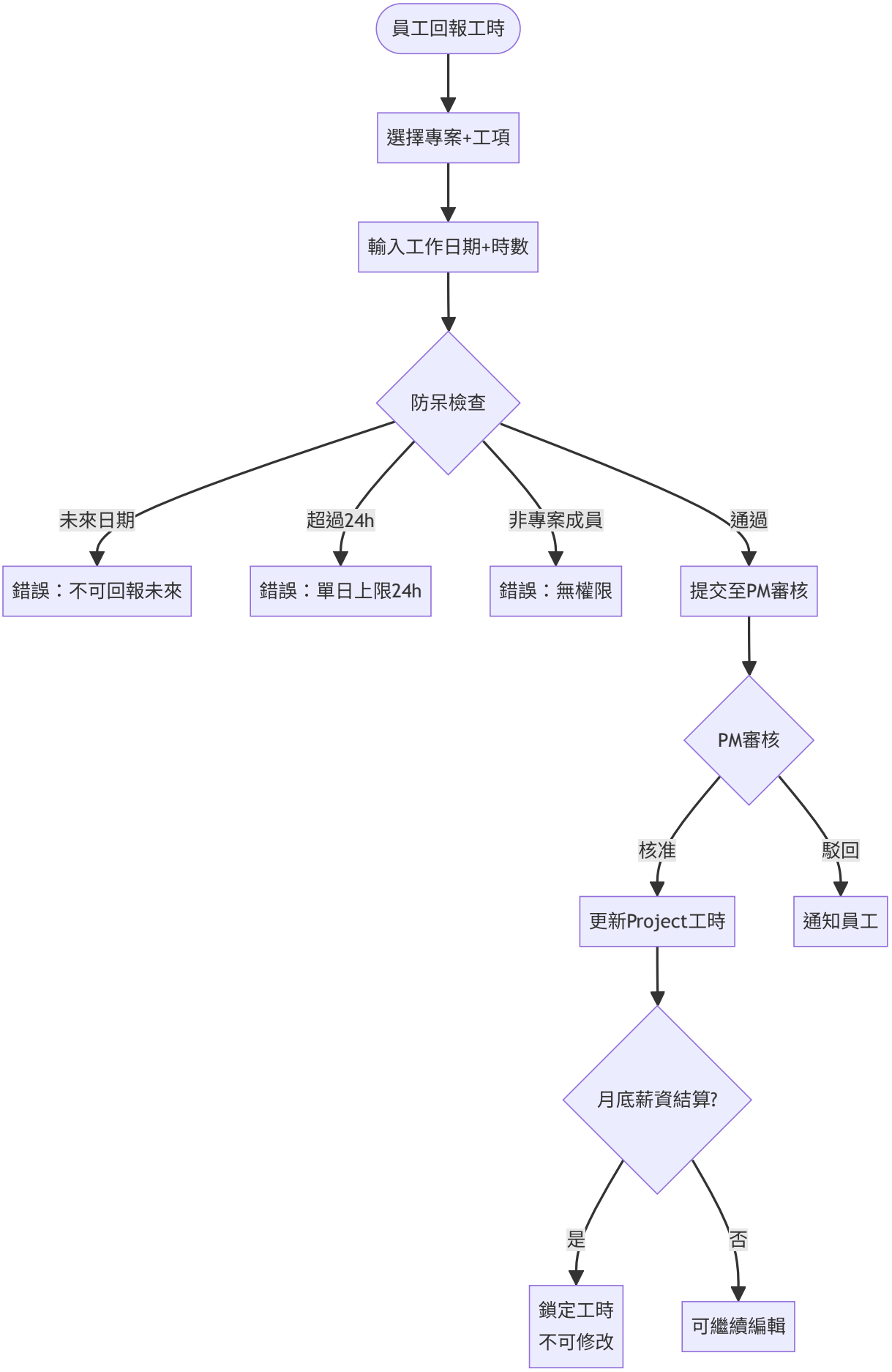
CROSS-002: 資料匯入/匯出功能

CROSS-003: 簽核流程整合設計

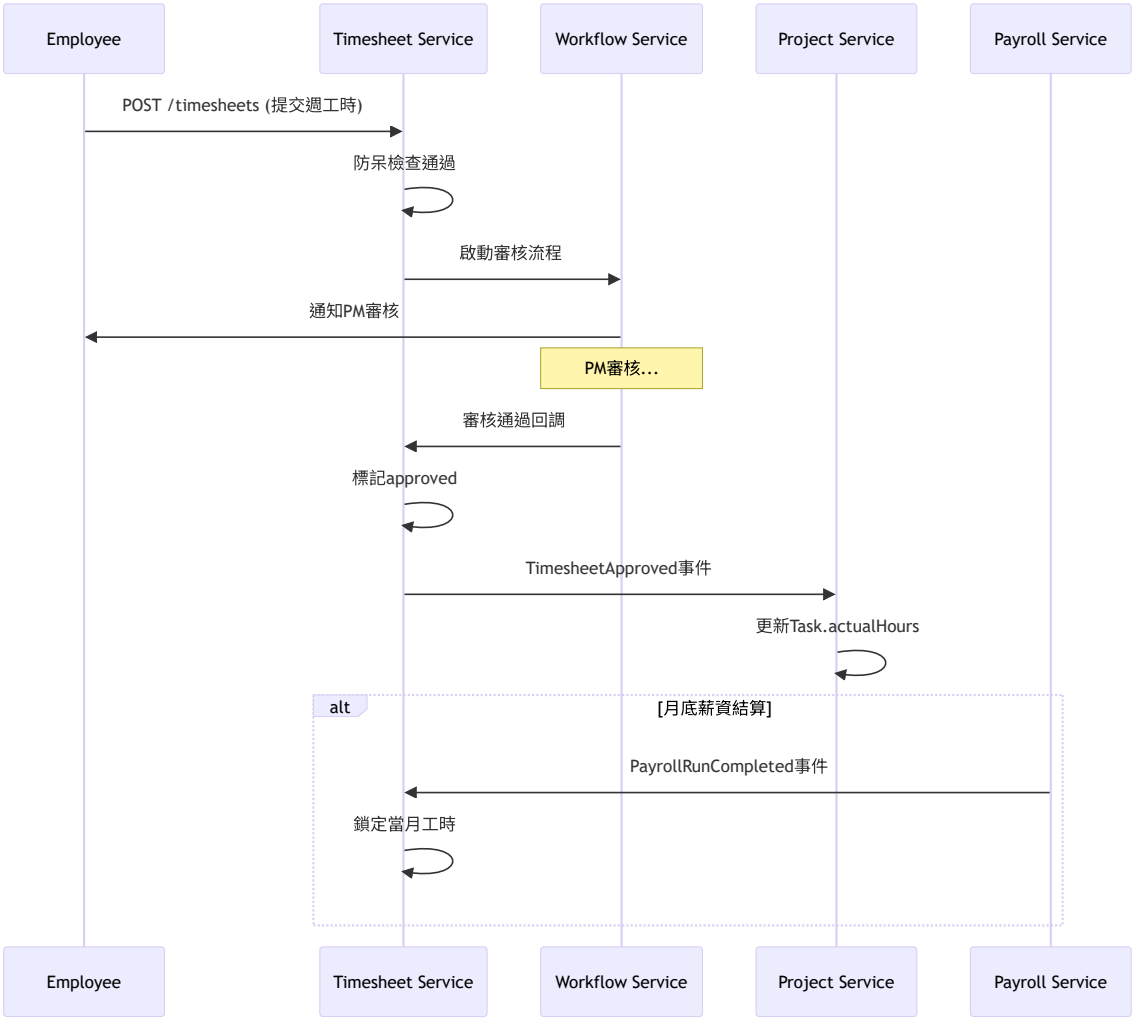
---

1. 工時管理服務 (07)

1.1 業務流程圖：工時回報與鎖定


員工回報工時
選擇專案+工項
輸入工作日期+時數
防呆檢查
未來日期
超過24h
非專案成員
通過
錯誤：不可回報未來
錯誤：單日上限24h
錯誤：無權限
提交至PM審核
PM審核
核准
駁回
更新Project工時
通知員工
月底薪資結算?
是
否
鎖定工時
不可修改
可繼續編輯

1.2 循序圖：工時審核與專案成本更新



1.3 業務案例：專案人力成本追蹤

情境: 追蹤XX銀行專案11月人力成本

```
專案成員工時統計（11月）：
張三 (Tech Lead): 120小時 × 291.67元/h = 35,000元
李四 (Developer): 100小時 × 250元/h = 25,000元
王五 (Developer): 80小時 × 208.33元/h = 16,667元
總計：76,667元

更新Project.actualCost累計：
10月累計：350,000元
11月新增：76,667元
總累計：426,667元
預算使用率：42.67%（總預算1,000,000元）
```

---

2. 簽核流程服務 (11) - CROSS-003

2.1 完整功能設計

代理人機制

```
UserDelegation {
  delegationId: UUID
  delegator: UUID (委託人)
  delegate: UUID (代理人)

  startDate: Date
  endDate: Date

  delegationScope: DelegationScope (ALL, SPECIFIC_TYPES)
  approvalTypes: List<String> (若SPECIFIC_TYPES)

  isActive: Boolean
}

// 範例：主管出國，指定代理人
delegator: 李經理
delegate: 王副理
startDate: 2025-12-10
endDate: 2025-12-20
delegationScope: ALL
```

催辦提醒機制

```
定時Job每日檢查：
- 待辦超過3天 → 發送第一次提醒
- 待辦超過5天 → 發送第二次提醒+通知上級
- 待辦超過7天 → 升級至更高層主管
```

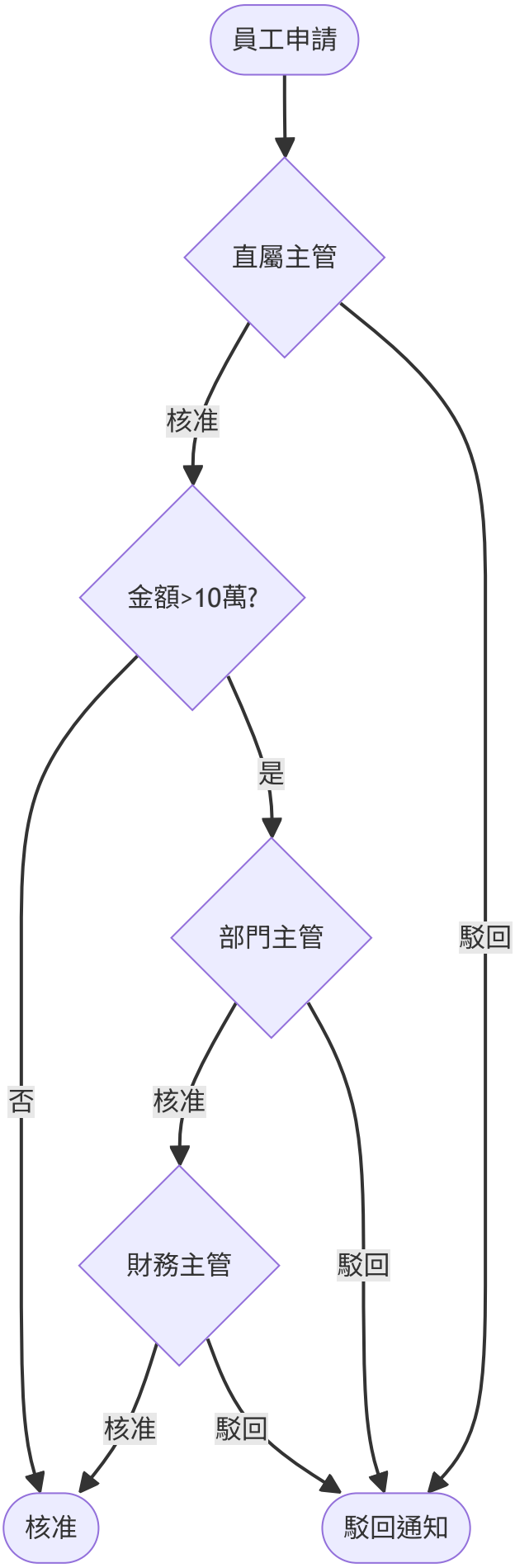
批次簽核

```
POST /api/v1/workflows/tasks/batch-approve
Request: {
  "taskIds": ["uuid1", "uuid2", "uuid3"],
  "comments": "批次核准"
}

Response: {
  "approved": 3,
  "failed": 0
}
```

可視化流程設計器（未來擴充）

- 拖拉式節點設計
- 支援條件分流（if-else邏輯）
- 支援平行會簽（所有人核准/任一人核准）
- 技術選型建議：使用bpmn.js或自行開發React組件

2.2 業務流程圖：多級簽核範例


員工申請
直屬主管
核准
駁回
金額>10萬?
是
否
部門主管
核准
駁回
財務主管
核准
駁回
核准
駁回通知

3. 通知服務 (12) - CROSS-001

3.1 完整架構設計

通知偏好設定

```
UserNotificationPreference {
  userId: UUID

  channels: Map<NotificationType, List<Channel>>
  // 範例：
  // LEAVE_APPROVED → [EMAIL, IN_APP]
  // PAYSLIP_GENERATED → [EMAIL]
  // OVERTIME_ALERT → [EMAIL, PUSH, LINE]

  quietHours: QuietHours (勿擾時段)
  language: String (zh-TW, en-US)
}

QuietHours {
  enabled: Boolean
  startTime: Time (22:00)
  endTime: Time (08:00)
}
```

企業通訊軟體整合

Microsoft Teams整合:

```java
@Service
public class TeamsNotificationService {

    public void sendToChannel(String webhookUrl, String message) {
        MessageCard card = MessageCard.builder()
            .title("系統通知")
            .text(message)
            .themeColor("0078D7")
            .build();

        restTemplate.postForEntity(webhookUrl, card, Void.class);
    }
}
```

LINE Notify整合:

```
POST https://notify-api.line.me/api/notify
Headers:
  Authorization: Bearer {LINE_ACCESS_TOKEN}
Body:
  message=您的請假申請已核准
```

Slack整合:

```
POST https://hooks.slack.com/services/{WEBHOOK_PATH}
Body: {
  "text": "系統通知",
  "blocks": [...]
}
```

3.2 事件訂閱清單（關鍵）

業務事件	通知對象	渠道	優先級
LeaveApplied	主管	Email+系統	NORMAL
LeaveApproved	申請人	Email+推播	NORMAL
OvertimeLimitExceeded	HR+主管	Email+系統	HIGH
PayslipGenerated	員工	Email	NORMAL
PasswordExpiringSoon	使用者	Email+系統	NORMAL
ProjectBudgetAlert	PM+專案總監	Email+Teams	URGENT
InsuranceEnrollmentCompleted	員工	Email	LOW

3.3 通知範本範例

```
【範本：LEAVE_APPROVED】
主旨：請假申請核准通知
內容：
親愛的 {{employeeName}}，

您申請的{{leaveType}}已獲核准：
- 假別：{{leaveTypeName}}
- 期間：{{startDate}} ~ {{endDate}}
- 天數：{{totalDays}}天
- 核准人：{{approverName}}
- 核准時間：{{approvedAt}}

剩餘假期餘額：{{remainingDays}}天

此為系統自動通知，請勿直接回覆。
```

---

4. 資料匯入/匯出 (CROSS-002)

4.1 批次匯入設計

Excel範本格式

```
員工批次匯入範本.xlsx:

Column A: 員工編號* (必填)
Column B: 姓名* (必填)
Column C: 身分證號* (必填，加密儲存)
Column D: Email* (必填)
Column E: 手機
Column F: 部門代碼*
Column G: 職稱*
Column H: 月薪*
Column I: 到職日*
...

備註：
- 標*為必填欄位
- 身分證號格式：A123456789
- Email須唯一
- 部門代碼需先建立
```

匯入API

```
POST /api/v1/employees/import
Content-Type: multipart/form-data

Request:
  file: employees.xlsx
  dryRun: true (預覽模式，不實際匯入)

Response:
{
  "totalRows": 50,
  "validRows": 45,
  "invalidRows": 5,
  "errors": [
    {
      "row": 3,
      "field": "email",
      "error": "Email重複"
    }
  ],
  "preview": [...]
}

若dryRun=false，則實際執行匯入
```

4.2 政府申報格式匯出

勞保局加退保申報XML

```
<?xml version="1.0" encoding="UTF-8"?>
<enrollment_report>
  <unit_code>12345678</unit_code>
  <report_month>2025-11</report_month>
  <enrollments>
    <enrollment>
      <id_number>A123456789</id_number>
      <name>張三</name>
      <enrollment_date>2025-11-01</enrollment_date>
      <insured_salary>48200</insured_salary>
    </enrollment>
  </enrollments>
</enrollment_report>
```

4.3 Excel匯出（通用）

```
GET /api/v1/employees/export?format=xlsx&filters=...

Response Headers:
Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
Content-Disposition: attachment; filename="employees_20251203.xlsx"

Body: (Excel binary)
```

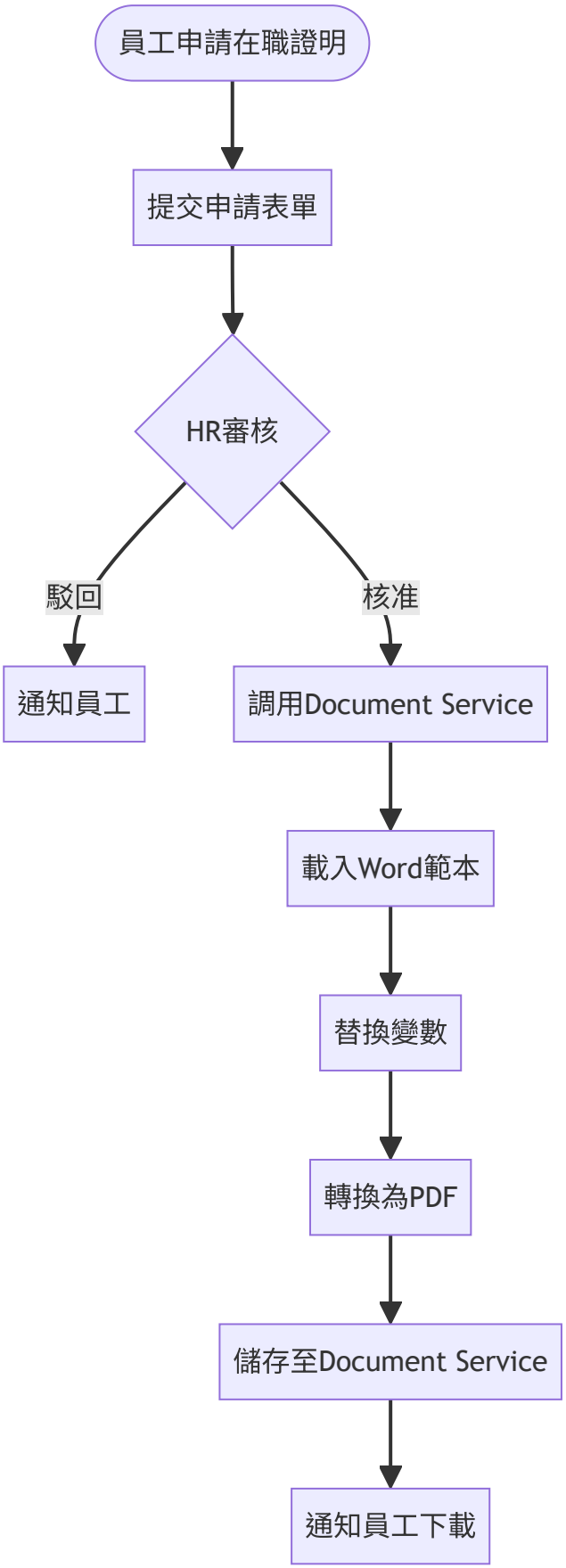
---

5. 文件管理服務 (13)

5.1 文件分類與權限

```
文件類型與可見性：

EMPLOYEE_CONTRACT (員工合約):
  - 擁有者：該員工+HR
  - 儲存位置：/storage/contracts/{employeeId}/

PAYSLIP (薪資單):
  - 擁有者：該員工+財務
  - 加密：AES-256
  - 密碼保護：員工身分證後4碼

EMPLOYEE_PHOTO (員工照片):
  - 擁有者：該員工+全公司（通訊錄）
  - 大小限制：2MB

POLICY_DOCUMENT (政策文件):
  - 擁有者：HR
  - 可見性：全公司
```

5.2 業務流程：產生在職證明



6. 報表分析服務 (14)

6.1 CQRS讀模型更新流程

6.2 關鍵報表API

人力盤點報表

```
GET /api/v1/reports/hr/headcount
Response: {
  "asOfDate": "2025-12-03",
  "totalEmployees": 150,
  "byDepartment": [
    {
      "department": "研發部",
      "count": 50,
      "avgSalary": 60000
    }
  ],
  "byEmploymentType": {
    "FULL_TIME": 140,
    "CONTRACT": 8,
    "INTERN": 2
  },
  "newHires": 5,
  "terminations": 2,
  "turnoverRate": "1.33%"
}
```

專案成本分析報表

```
GET /api/v1/reports/project/cost-analysis?projectId={id}
Response: {
  "project": "XX銀行專案",
  "budget": 10000000,
  "actualCost": 3500000,
  "utilizationRate": "35%",
  "profitMargin": "65%",
  "monthlyTrend": [
    {"month": "2025-11", "cost": 350000}
  ],
  "topCostMembers": [
    {"name": "張三", "hours": 120, "cost": 35000}
  ]
}
```

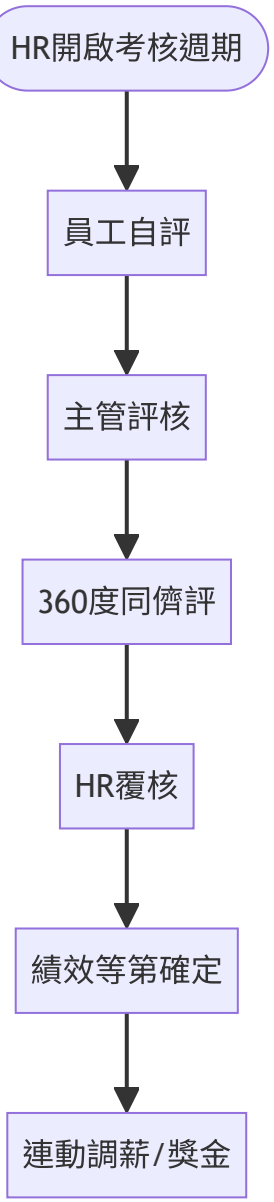
6.3 視覺化儀表板範例

高階主管儀表板配置:

```
{
  "dashboardName": "CEO Dashboard",
  "widgets": [
    {
      "type": "KPI_CARD",
      "title": "在職人數",
      "dataSource": "employee_count",
      "value": 150
    },
    {
      "type": "KPI_CARD",
      "title": "本月離職率",
      "dataSource": "turnover_rate",
      "value": "1.33%"
    },
    {
      "type": "LINE_CHART",
      "title": "月度人力成本趨勢",
      "dataSource": "monthly_labor_cost",
      "data": [...]
    },
    {
      "type": "PIE_CHART",
      "title": "部門人數分布",
      "dataSource": "headcount_by_department"
    }
  ]
}
```

7. 績效/招募/訓練服務 (08-10)

7.1 績效考核流程



7.2 招募漏斗分析

```
招募漏斗（某職缺）：
應徵者：100人
  ↓ 履歷篩選 (50%淘汰)
面試邀約：50人
  ↓ 面試 (60%淘汰)
Offer發放：20人
  ↓ 接受率 (70%)
到職：14人
  ↓ 試用期通過率 (90%)
留任：12人

招募效率指標：
- Time to Hire: 平均45天
- Cost per Hire: 平均50,000元
- Offer Acceptance Rate: 70%
```

7.3 訓練時數追蹤

```
法定訓練時數要求：
- 一般員工：12小時/年
- 管理職：20小時/年
- 新進員工：40小時（首年）

系統追蹤：
張三（一般員工）：
  已完成：8小時
  剩餘：4小時
  狀態：⚠️ 需注意（剩餘4個月）
```

---

8. 操作手冊補充

8.1 HR月度作業檢查清單

```
每月1日：
☐ 檢查上月考勤是否全部結算
☐ 確認所有請假/加班已審核

每月5日前：
☐ 執行月度薪資計算
☐ 檢查薪資計算異常
☐ 核准薪資
☐ 產生銀行薪轉檔
☐ 發送薪資單Email

每月10日：
☐ 檢查保險加退保申報
☐ 上傳勞保局e化系統

每月底：
☐ 檢查特休到期提醒
☐ 檢查合約到期提醒
☐ 檢查證照到期提醒
```

補充文件結束

涵蓋服務: 07工時管理、08-10績效/招募/訓練、11簽核流程、12通知、13文件、14報表分析
修訂日期: 2025-12-03
修訂人: SA